

Flow Matching Before a Frozen Linear Classifier

Reshaping frozen features for a classifier that never learns again

Yousef Shihade · Mira Bitar | University of Haifa · 2026

In Stage 3 we place a flow-matching layer between the frozen image feature and the linear classifier we trained in Stage 1, and we hold that classifier fixed. The flow is initialised so that it is exactly the identity, which means the untrained system is bit-for-bit the Stage 1 linear probe, and every result in this report is caused by training the flow and nothing else. We compare two training strategies against that probe and against each other, over 180 training runs.

+0.1 to +2.6 pp

The flow helps, modestly. Stage 2 predicted this: its headroom rule says the flow recovers what training the classifier would have gained, and here the classifier is already trained.

+0.13 pp, $p = 0.80$

The two strategies are indistinguishable on accuracy, but not on robustness. Classifier-guided training survives a 10× larger learning rate ($p = 0.048$).

−0.02 pp, $p = 0.97$

Unfreezing the classifier changes nothing, which is what licenses the frozen-classifier framing of everything else.

1 Stage 3 at a glance

$$z \rightarrow [\text{FM}, T = 4 \text{ Euler steps}] \rightarrow \hat{z} \rightarrow [\text{frozen } W, b] \rightarrow \text{logits}$$

The encoder is frozen, as in Stage 1. The classifier is the Stage 1 probe, trained and then frozen. The flow is the only trainable component, and because it starts as the exact identity, at epoch 0 the system *is* the Stage 1 linear probe: $\Delta = 0.00$ pp, with identical predictions on every test image.

Step	What it does	Runs	Time
01 Setup	Recover and freeze the Stage 1 classifier; verify the identity initialisation	—	—
02 Strategy 1	End-to-end rolled-out classification, plus a regularisation sweep	81	32 min
03 Strategy 2	Classifier-guided targets, plus a four-knob sweep	99	16 min
04 Evaluation	The consolidated comparison; retrain nothing	—	—
05 Visualisation	Joint PCA, and geometry measured in the full feature space	6	81 s
06 Joint	Unfreeze the classifier, with a classifier-only control	45	13 min

Table 1. What was run.

Steps, seeds	size,	$T = 4$ Euler steps · $K = 10$ · seeds $\{0, 1, 2\}$, the same k-shot subsets (and therefore the same images) as Stages 1 and 2
Features		Raw, with no L2 normalisation. This is the opposite of Stage 2 and the one decision Stage 3 inherits differently; Section 3 measures why
Velocity network		Stage 2's design unchanged (2×512 hidden, SiLU, t concatenated), with the output layer zeroed
Checkpointing		On validation accuracy, Stage 1's own rule, so the baseline and both strategies are selected identically

Reproducibility: 52 tests pass across Stages 2 and 3. Every figure and table in this report is generated by a tracked notebook and backed by a tracked CSV, and each step folder carries an `_artifact_manifest.csv` listing exactly what it wrote. Stages 1 and 2 are not modified, which we verified against git.

2 What Stage 3 asks

Stage 1 built a linear probe: freeze the encoder, extract features z , and train $s = Wz + b$ on $K = 10$ images per class. That is as good as a linear rule on those features can be. Stage 3 asks a sharper question. The features are frozen and possibly badly shaped; can we learn to reshape them so that the classifier we already have does better?

Four constraints turn this into a real experiment. The classifier is trained first, exactly as in Stage 1, and then frozen; it never learns again, so it cannot adapt to whatever the flow does, and the flow has to move features toward what the classifier already wants. The flow is initialised close to identity, so that before training the whole system behaves like the original probe, which is what gives “ Δ vs. probe” a meaning. One representative encoder per dataset, one training-set size and one T are used throughout. And three things are compared per dataset: the Stage 1 probe, end-to-end rolled-out training, and classifier-guided training.

	Strategy 1 · end-to-end	Strategy 2 · classifier-guided
Loss	$\text{CE}(W\hat{z} + b, y)$	standard FM: $\text{MSE}(v(z_t, t), u)$
Gradient path	through all T Euler steps	never through the rollout
Classifier's role	it is the loss	it builds a target, then ordinary FM
Main risk	backprop through T composed steps	targets go stale between refreshes

Table 2. The two strategies are different answers, not two tunings of one. Strategy 2 in one line: roll $z \rightarrow \hat{z}$, take gradient steps on $\hat{z}$ in feature space to get a version the classifier likes better, then train the flow to transport $z \rightarrow \hat{z}'$ with a completely standard FM objective. The classifier says where to go; the FM machinery does the going. Strategy 2's loss averages over the feature dimension as well as the batch, unlike Stage 2's summed convention, so the two are not on the same numeric scale.

Combination	Classes	K=10 train	Probe acc.	Role
ResNet-18 / DTD	47	470	52.04%	main
DINOv2 / FGVC-Aircraft	100	1000	51.54%	main
ResNet-18 / FGVC-Aircraft	100	1000	27.87%	extra

Table 3. Datasets and encoders, carried unchanged from Stage 1. The third combination goes beyond the two main combinations. We include it because Stage 1 measured its features as the most entangled of the nine (own-vs-other cosine margin 0.041), which makes it the most informative test of whether a flow can rescue a representation a linear classifier struggles with.

3 The one decision Stage 3 had to make differently

Stage 2 L2-normalised its features. Stage 3 must not, and we measured the cost rather than assuming it. Stage 2 normalised because its targets were class prototypes, which are unit-norm by construction, and its decision rule was cosine similarity, which is scale-invariant. Stage 3’s frozen classifier was trained on raw features at norms of 24–50, and an affine rule is not scale-invariant: $W(z/\|z\|) + b$ rescales Wz but leaves b untouched, so the decision boundary moves relative to the data.

Encoder / dataset	mean $\ z\ $	raw	L2-normalised	cost
ResNet-18 / DTD	23.7	52.04%	47.55%	−4.49 pp
DINOv2 / FGVC-Aircraft	49.7	51.54%	46.19%	−5.34 pp
ResNet-18 / FGVC-Aircraft	28.9	27.87%	24.44%	−3.43 pp

Table 4. What normalising would cost the frozen probe.

This is not a collapse, since normalising preserves each feature’s direction, where most of the class information lives, but it is a free loss of 3.4–5.3 pp on every combination. The DINOv2 row is the sharpest form of the argument: its norms are already tightly clustered (46.9–52.0), so normalising is close to dividing every feature by the same constant, and it still costs the most of the three. A near-uniform rescale is not harmless to an affine classifier. The same fact is why the identity initialisation is load-bearing rather than book-keeping. A flow initialised the ordinary way would displace features on the very first forward pass, and the classifier in front of it has no slack for that.

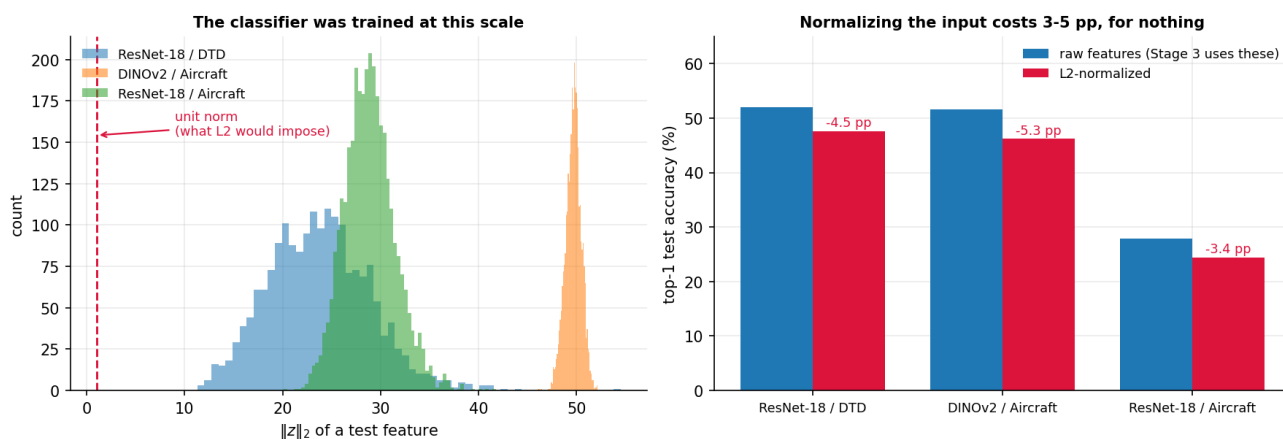


Figure 1. Feature-norm distributions per combination (left; the red line is the unit norm L2 normalisation would impose), and the measured accuracy cost of feeding the frozen classifier L2-normalised features instead of raw ones (right).

4 Recovering and freezing the classifier STEP 01

No flow is trained in this step. It establishes four things everything downstream stands on, each measured rather than assumed.

The first was a blocker: Stage 1 never saved the classifier. It saved accuracies, training curves and per-image predictions, but not W and b ; the trained layer went out of scope, because Stage 1 only reported numbers and Stage 2 used prototypes instead. Since Stages 1 and 2 are finished, we modified neither. Stage 3 carries its own `train_frozen_classifier`, which reproduces Stage 1’s loop and additionally returns the weights. The hyperparameters are not duplicated: `ProbeConfig` is imported from Stage 1, so the learning rate, weight decay,

batch size, epoch budget and checkpoint rule keep exactly one definition in the project. Duplicated logic is normally a liability, and two things stop it being one here. This step asserts that the re-derived probes reproduce Stage 1 exactly, and an 11-test equivalence suite runs both implementations against each other on accuracy, predictions, selected epoch and every training curve.

Check	Result
Re-derived probes vs. Stage 1's published accuracies (9 runs)	max $ \Delta = 0.00$ pp
Test predictions, per image	identical; 0 of 16,920 differ
Identity init: max $ \hat{z} - z $ over all test features	0.0, bit-for-bit
Accuracy at initialisation vs. the probe	$\Delta = 0.00$ pp
Classifier parameters requiring grad	0
$W.grad$ after a full backward pass through the system	None

Table 5. What step 01 verified.

“Frozen” is mechanical, not a promise. `FrozenClassifier` sets `requires_grad=False` at construction and overrides `.train()` to refuse to leave eval mode, so a parent module calling `.train()` on the whole system cannot silently unfreeze it. Gradient must still flow *through* the layer, because Strategy 1 differentiates the loss back into the flow behind it; that is a different thing from the classifier’s own parameters being updated, and both properties are pinned by tests.

One consequence of starting at the exact identity is worth stating. Only the readout is zeroed; the hidden layers keep their random initialisation. A hidden layer’s gradient is proportional to the readout weight above it, so while the readout is exactly zero the hidden layers receive exactly zero gradient, and on the first optimiser step only the readout moves. We measured this: hidden $|grad| = 0.0$ at step 0, then $3.7e-03$ and $7.7e-03$ at steps 1 and 2, with the loss falling throughout. This is standard behaviour for a zero-initialised readout, and it is the price of starting exactly at identity rather than near it.

5 Strategy 1: end-to-end rolled-out training STEP 02 · 81 RUNS

For each training feature z we run the full $T = 4$ rollout to get $\hat{z}$, push it through the frozen classifier, and minimise $CE(W\hat{z} + b, y)$, backpropagating through the entire rollout and updating only the flow’s parameters. Both regularisers we implemented are swept at three strengths each: displacement $\|\hat{z} - z\|^2$ and velocity magnitude $\|v_k\|^2$. Velocities are recovered from the trajectory ($v_k = T(\hat{z}_{k+1} - \hat{z}_k)$) rather than by re-running the network, so the penalty is measured on exactly the states the loss saw. In total the step is 81 runs: 9 configurations $\times$ 3 combinations $\times$ 3 seeds, in about 32 minutes.

Is there anything left to optimise?

The loss is the frozen probe’s classification loss on the probe’s own training set, the same 470 or 1000 k-shot features it was fitted on in Stage 1. We measured it at identity, before any training.

Encoder / dataset	train acc	val acc	CE(train)	CE(val)	ratio
ResNet-18/DTD	98.44%	51.52%	0.327	1.823	7×
DINOv2/Aircraft	100.00%	52.11%	0.005	2.304	449×
ResNet-18/Aircraft	100.00%	27.67%	0.031	3.573	122×

Table 6. The Strategy 1 objective at identity, before training.

The objective is almost exhausted before training starts. That does not mean there is no gradient; the curves below show training cross-entropy driven down to 10^{-6} . It means that following the signal is memorisation. The real question Strategy 1 poses is therefore not whether the flow can reduce the loss (it can, completely) but how little it can be allowed to move before memorisation outweighs whatever real structure it finds.

Config	Δ vs. probe	sel. epoch	moved
main (lr 10^{-3})	$+0.15 \pm 0.85$ pp	9	12.7%
lr 10^{-4}	$+1.27 \pm 1.32$ pp	28	28.6%
lr 10^{-5}	$+1.21 \pm 0.91$ pp	131	16.4%
displacement 0.001 / 0.01 / 0.1	$+0.70$ / $+0.64$ / $+0.29$	107 / 140 / 105	4.6 / 2.5 / 0.9%
velocity 0.001 / 0.01 / 0.1	$+0.78$ / $+0.56$ / $+0.32$	101 / 118 / 68	4.7 / 2.5 / 1.0%

Table 7. The Strategy 1 sweep. "Moved" is the mean displacement $\|\hat{z} - z\|$ as a fraction of $\|z\|$.

The default learning rate turned out to be the worst choice we tested. A rate of 10^{-3} is the value Stages 1 and 2 both use, and carrying it over was the natural choice; it was right in Stage 2, where a flow trains from scratch against fixed targets. Here it is actively harmful. It drives a 657k–788k-parameter network onto 470–1000 features so fast that validation accuracy never rises, and the checkpoint rule salvages an epoch-9 model that has barely moved. Reducing the learning rate tenfold is worth about +1.1 pp, which is larger than anything the regularisation achieves. Constraining how fast the flow moves beats constraining how far.

What the flow is actually exploiting

The step's most interesting result runs against the obvious reading of Table 6. If the flow were harvesting the residue of the classification objective, the combination with the most unminimised loss should gain the most. The opposite happens.

Combination	CE(train) at identity	gain at lr 10^{-5}
ResNet-18/DTD	0.327 (most left to minimise)	+0.32 pp (smallest)
ResNet-18/Aircraft	0.031	+1.09 pp
DINOv2/Aircraft	0.005 (least left to minimise)	+2.21 pp (largest)

Table 8. Remaining training loss anti-predicts the gain.

Whatever the flow is exploiting, it is not the residue of the classification objective. Displacement tells the same story from the other side: across all 81 runs, how far the flow moved features correlates positively with the gain (Spearman $\rho = +0.54$, $p < 0.001$; +0.70 on DINOv2/Aircraft). The flow has to move features to help; it just has to get there slowly. What we do not claim is an explanation of the ordering. With three combinations it is impossible to say what does explain it, since DTD differs from the two Aircraft combinations in dataset, class count (47 vs. 100) and training-set size (470 vs. 1000) simultaneously. Stage 2's headroom rule earned its claim because nine cells lined up behind it; three do not support a rule here.

Strategy 1: training accuracy is pinned near 100% from the start; validation peaks immediately and decays

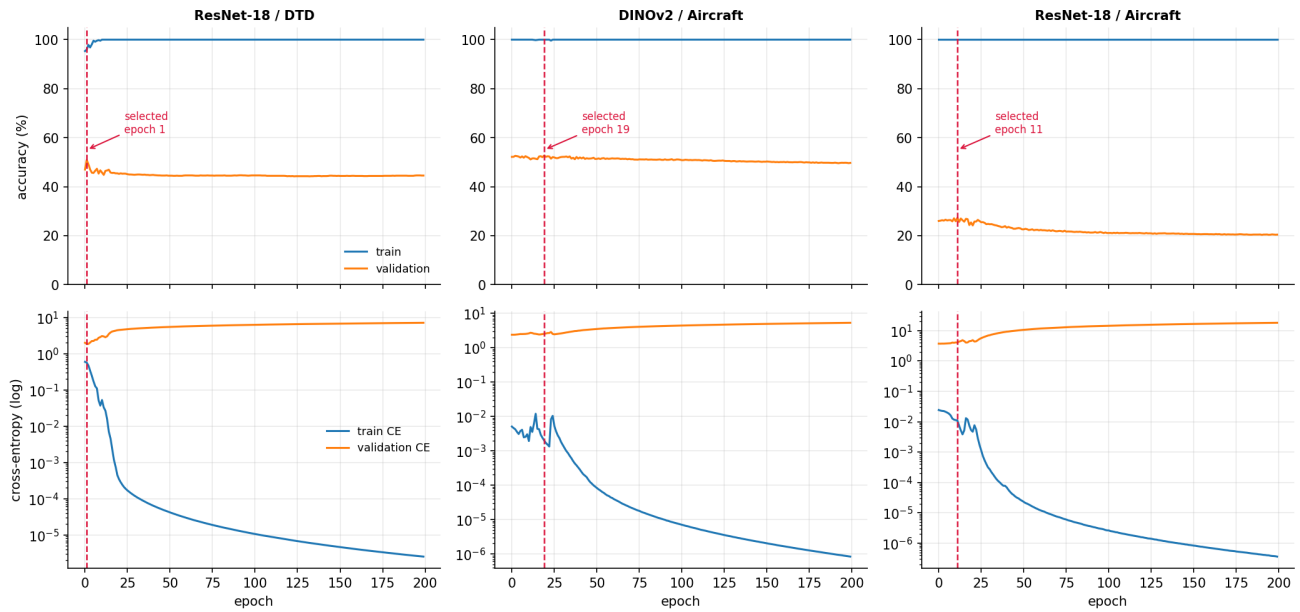


Figure 2. Strategy 1 training curves. Training accuracy is pinned at $\sim 100\%$ for all 200 epochs because it starts there. Training cross-entropy falls five orders of magnitude, all of it on data already classified correctly, while validation cross-entropy rises monotonically from epoch 0; there is no regime in which both improve.

Validation accuracy peaks within the first 20 or so epochs and decays thereafter, so checkpoint selection on validation accuracy is not a refinement here but what makes the method work at all. It is also Stage 1's own rule, which keeps the comparison against the probe consistent.

6 Strategy 2: classifier-guided targets STEP 03 · 99 RUNS

Strategy 2 follows six steps:

1. Roll z through the current flow to get $\hat{z}$.
2. Compute the frozen classifier's loss on $\hat{z}$.
3. Build an improved $\hat{z}'$ by descending that loss in feature space, $\hat{z}' = \hat{z} - \eta \nabla_{\hat{z}} \text{CE}$, unit-normalised so that η is the distance actually travelled.
4. Take z as source and $\hat{z}'$ as target, detached; this is what makes it an ordinary FM update rather than a second end-to-end objective.
5. Train with standard FM: $t \sim \mathcal{U}(0, 1)$, $z_t = (1 - t)z + t\hat{z}'$, $u = \hat{z}' - z$, $\mathcal{L} = \text{MSE}(v(z_t, t), u)$.
6. Recompute targets every `refresh_every` epochs, as the flow changes.

This loss averages over the feature dimension as well as the batch, unlike Stage 2's summed $\|\cdot\|^2$ convention (Section 2), so the two are not on the same numeric scale — only the shape of each training curve is comparable, not the raw magnitude.

The flow can never beat the targets it is aimed at, so we measured those first.

Encoder / dataset	base acc	base CE	target acc ($\eta=1$)	target CE ($\eta=1$)
ResNet-18/DTD	96.38%	0.594	99.15%	0.316
DINOv2/Aircraft	100.00%	0.005	100.00%	0.001
ResNet-18/Aircraft	100.00%	0.022	100.00%	0.001

Table 9. What the guidance produces, on the training set.

On two of the three combinations the targets start at 100% accuracy and stay there. The probe already classifies its own k-shot training set perfectly, so there is no correctness for the guidance to add; what the gradient still does is collapse the target cross-entropy toward zero by pushing features into higher-confidence regions. Strategy 2 is therefore not “move features so the classifier gets them right”. It is a margin-increasing transform that moves features so the classifier is more certain about answers it already gives. That it still generalises, by +1.3 to +1.9 pp, is the interesting part.

The four-knob sweep

Strategy 2 exposes exactly four things to experiment with, and we varied each around the main configuration: 11 configurations $\times$ 3 combinations $\times$ 3 seeds, 99 runs in about 16 minutes.

Config	Δ vs. probe	sel. epoch	moved
main ($\eta=1$, 1 step, refresh 1, normalised)	+1.40 $\pm$ 0.79 pp	119	17.1%
$\eta = 0.5$	+1.53 $\pm$ 0.66 pp	109	12.0%
$\eta = 2$	+1.54 $\pm$ 0.72 pp	91	19.5%
$\eta = 4$	+1.18 $\pm$ 0.83 pp	103	34.0%
3 target steps	+1.12 $\pm$ 0.84 pp	83	29.7%
refresh every 10	+1.82 $\pm$ 0.80 pp	116	11.3%
refresh every 50	+1.59 $\pm$ 0.76 pp	143	6.1%
unnormalised, $\eta = 10^3$	+0.16 $\pm$ 0.24 pp	99	2.1%
unnormalised, $\eta = 10^4$	+0.46 $\pm$ 0.46 pp	133	4.3%
lr 10^{-3}	+0.94 $\pm$ 0.71 pp	17	10.8%
lr 10^{-5}	+0.89 $\pm$ 0.80 pp	144	8.4%

Table 10. The Strategy 2 sweep. Green: best configuration. Red: the two unnormalised variants.

Four things stand out. Refreshing the targets less often is better: recomputing targets as the flow changes, every epoch gives +1.40 pp while every 10 epochs gives +1.82 pp. Recomputing constantly means chasing a target that moves with the flow chasing it, and letting it go stale for a few epochs makes the objective closer to stationary. Normalising the guidance step matters more than any other choice, +0.16/+0.46 pp unnormalised against +1.4 to +1.8 normalised, and the reason is measured: the raw gradient norm differs 100 $\times$ across combinations (7.4e -4 on DTD, 7.1e -6 on DINOv2), so one nominal step size means three very different effective steps. Step size is otherwise forgiving, with $\eta \in [0.5, 2]$ all landing within 0.15 pp of each other. And more target steps is worse: 3 steps moves features 30% of $\|z\|$ and loses 0.3 pp against 1 step. Overall, 91 of the 99 runs (92%) improve on the frozen probe, and all 11 configurations are positive on average.

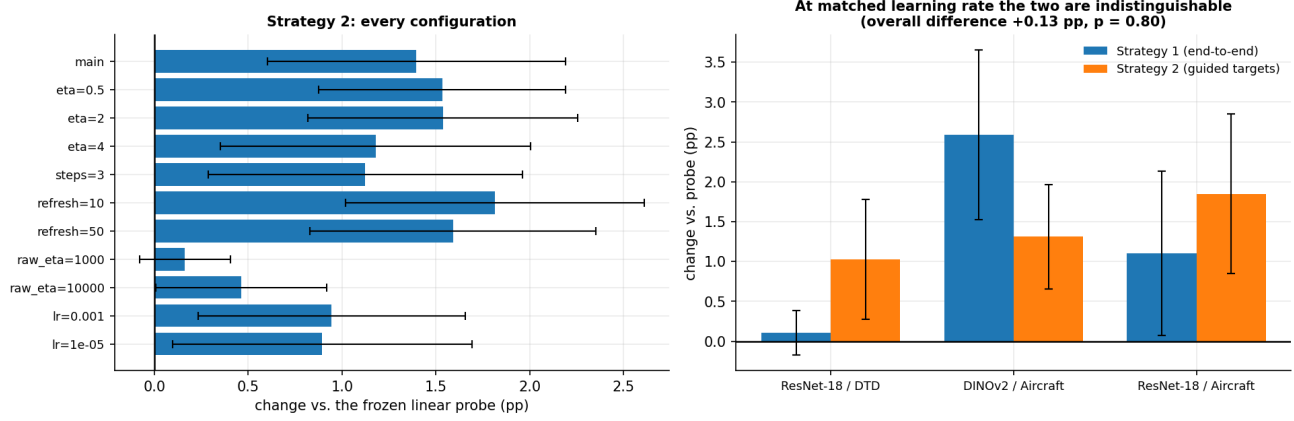


Figure 3. Every Strategy 2 configuration's Δ against the frozen probe (left), and the strategy comparison at matched learning rate (right; see Section 7).

7 Comparing the two strategies STEP 04

Nothing is retrained in this step; every number comes from the run tables steps 02 and 03 wrote.

Combination	Probe	Strategy 1	Δ Acc	Strategy 2	Δ Acc
ResNet-18/DTD	52.04%	52.15 1.15	$\pm$ +0.11	53.24 1.39	$\pm$ +1.21
DINOv2/Aircraft	51.54%	54.13 1.13	$\pm$ +2.59	53.73 1.53	$\pm$ +2.19
ResNet-18/Aircraft *	27.87%	28.97 0.74	$\pm$ +1.10	29.92 0.60	$\pm$ +2.05

Table 11. Top-1 test accuracy, $K = 10$, $T = 4$, mean $\pm$ sample std over 3 seeds. Bold: the better strategy per row. * Beyond the two main combinations.

Every combination improves on average under both objectives: Strategy 1 by +1.27 pp overall and Strategy 2 by +1.82 pp. Steps 02 and 03 swept 9 and 11 configurations respectively, and picking the winner by test accuracy would make this headline optimistic by construction, so the reported configuration is chosen by mean validation accuracy, which appears in no reported figure. As it happens, validation and test independently pick the same configuration for both strategies ($lr=1e-4$ and $refresh=10$), so the numbers above are not a product of test-set selection.

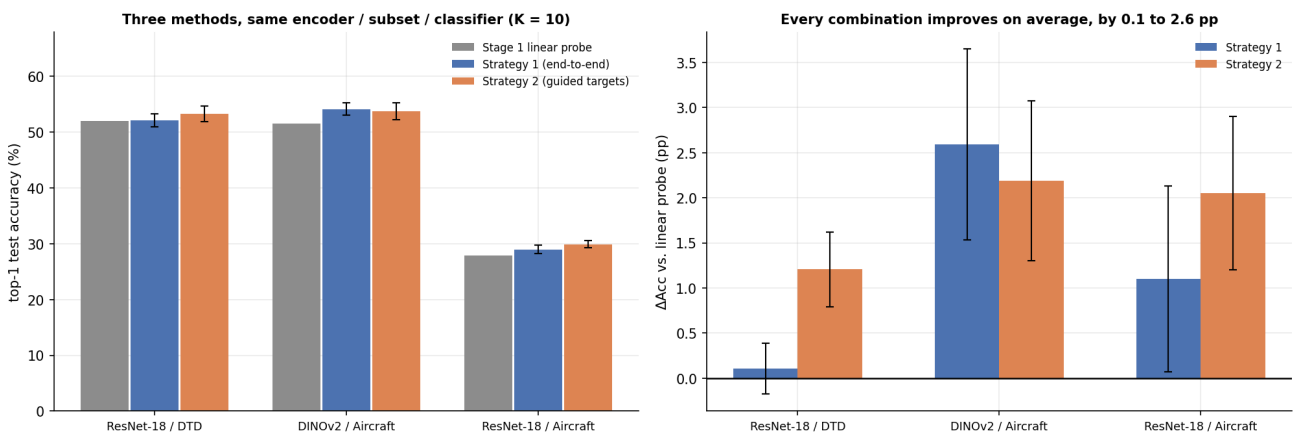


Figure 4. The three methods per dataset (left) and Δ Acc against the Stage 1 linear probe (right), error bars over 3 seeds.

Do the two strategies actually differ?

Strategy 2's main configuration already used $\text{lr } 10^{-4}$, carrying step 02's finding, while Strategy 1's main configuration used 10^{-3} . Comparing those two directly would compare Strategy 2 with step 02's lesson against Strategy 1 without it, a difference in tuning rather than in objective. Holding the learning rate fixed isolates the objective.

lr	Strategy 1	Strategy 2	difference	p
10^{-3}	+0.15	+0.94	+0.79	0.048
10^{-4}	+1.27	+1.40	+0.13	0.802
10^{-5}	+1.21	+0.89	−0.31	0.451

Table 12. Mean Δ vs. probe (pp) for each strategy at each learning rate.

On accuracy the two are indistinguishable, +0.13 pp at $p = 0.80$, and they trade wins per combination. Even best configuration against best configuration, selected after the fact on both sides so that the comparison flatters both, the gap is +0.55 pp at $p = 0.30$. On robustness they clearly and significantly differ ($p = 0.048$ at 10^{-3}). Strategy 1's validation cross-entropy rises far faster on all three combinations, so it has to be stopped much earlier: selected epochs 4/66/35 against 101/117/198. The mechanism is the objective itself. Strategy 1 optimises the classification loss directly, so a large step memorises training labels within a handful of epochs, whereas Strategy 2 optimises an MSE regression onto a target vector, and the labels reach it only through where that target was placed, so the same step size cannot memorise them nearly as fast.

DOES CLASSIFIER-GUIDED TRAINING BEAT END-TO-END TRAINING?

No. Classifier-guided training does not beat end-to-end rolled-out training on accuracy. It is substantially more robust to the learning rate, and it improves in a higher fraction of runs (91/99 against 67/81).

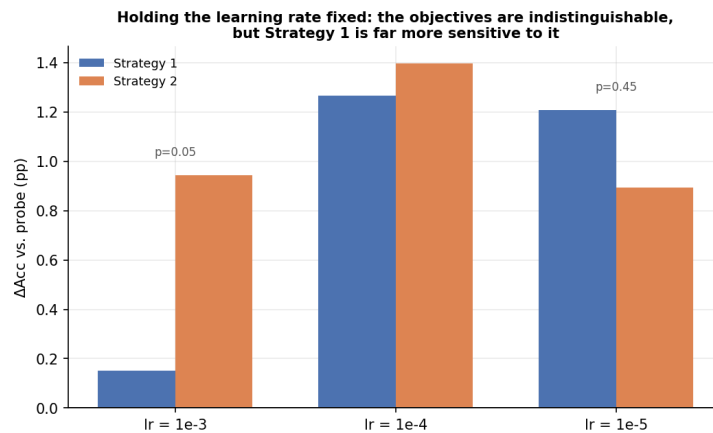


Figure 5. Objective versus tuning: the two strategies at each learning rate.

8 What the flow does to the feature space STEP 05

Figure 6 shows three panels for one combination, original z , Strategy 1 $\hat{z}$ and Strategy 2 $\hat{z}$, on the same test images with the same class colours, and one PCA fitted on the concatenation of all three sets, so the panels are directly comparable rather than three independent rotations. We use PCA rather than t-SNE because it is deterministic, with no perplexity or seed to justify, and because "compute the embedding jointly" has an unambiguous meaning for a linear method. Classes are picked by an even stride through the sorted class list, not by hand. These plots should be read as subset diagnostics: 6 classes, one seed, 199–240 test images, and two components capturing only 20.7–33.8% of the variance. That last number is exactly why we also measure the geometry in the full feature space below.

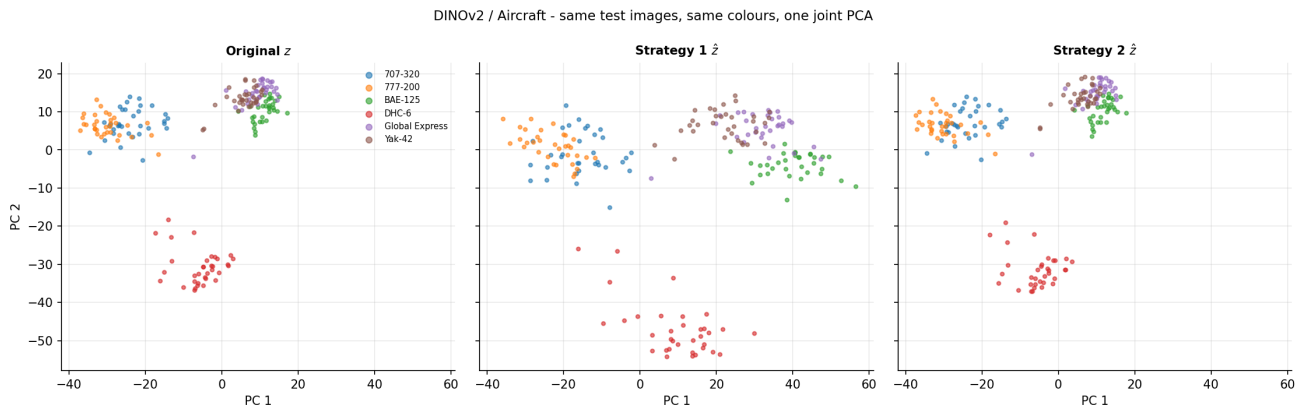


Figure 6. DINOv2 / FGVC-Aircraft. Strategy 1 visibly stretches the whole space; Strategy 2’s panel is almost indistinguishable from the original, for an equal-or-better result.

DINOv2 / Aircraft	displacement $\ \hat{z}-z\ $	Δ accuracy (6 cls)
Strategy 1	34.94	+1.01 pp
Strategy 2	4.60	+1.51 pp

Table 13. The clearest thing the pictures show: an equal-or-better result with roughly 8× less movement. How far the flow moves features is not what produces the gain.

Measuring the geometry properly

A 2-D projection cannot say whether class structure improved. Two quantities measured in the full feature space can, and they disagree in an informative way.

Combination	Strat.	Δ acc (6 cls)	logit margin	Δ sep.
ResNet-18/DTD	1	−1.25	0.997×	+0.0046
	2	−0.42	1.92×	+0.0445
DINOv2/Aircraft	1	+1.01	1.58×	−0.0739
	2	+1.51	1.64×	+0.0059
ResNet-18/Aircraft	1	+2.01	1.45×	+0.0429
	2	+3.52	1.62×	+0.0103

Table 14. Geometry in the full feature space for the six-class subsets. Logit margin is the ratio of the top-minus-runner-up logit gap after the flow to before; separability is Stage 1’s own-minus-other cosine similarity.

The logit margin tracks the result. The frozen classifier is linear, so its decision is entirely the gap between the top logit and the runner-up, and five of the six cases widen that gap by 1.45–1.92×. The single exception is Strategy 1 on DTD at 0.997×, no widening at all and in fact a hair narrower, which is precisely the combination where Strategy 1 delivered nothing on the full test set (+0.11 pp, one seed negative). The diagnostic and the headline agree.

Class separability does not track the result. The own-minus-other cosine similarity, the training-free measure from Stage 1 that predicted the whole Stage 1 ordering, moves the wrong way where the gain is real: Strategy 1 on DINOv2/Aircraft loses 0.074 of separability while gaining accuracy. Stage 2 recorded the same lesson from the other direction, when flows contracted features toward the right prototype without separating them from the wrong ones and accuracy did not follow. Neither clustering in a projection nor class-mean geometry is the thing a linear classifier actually consumes; the margin is. This also explains why Strategy 2 needs so little movement. Its target vector already points along the direction that widens the margin, so it moves features a short distance in a

useful direction, whereas Strategy 1 has no such guidance and must discover a direction by backpropagating through the rollout.

9 Unfreezing the classifier STEP 06 · 45 RUNS

RESULT

Unfreezing the classifier makes no measurable difference: -0.02 pp, $p = 0.97$. The classifier's extra freedom is available and simply goes unused. Keeping it frozen, which is what makes every other number in Stage 3 interpretable, costs nothing.

There is a confound to resolve before that result can be read. If flow plus unfrozen classifier beats the probe, there are two explanations: the flow is helping, or the classifier just got more training. Stage 1 trained the probe for up to 200 epochs with validation checkpointing, so it should be at its ceiling, but "should" is not a measurement. This step therefore runs a classifier-only control that continues training the classifier from Stage 1's weights with the same optimiser, budget and checkpoint rule and no flow at all. Without this control the extension cannot be interpreted. The step is 5 configurations $\times$ 3 combinations $\times$ 3 seeds, 45 runs in about 13 minutes.

Combination	clf. only	joint	flow tributes	con- fined	frozen S1	frozen S2
ResNet-18/DTD	+0.02	+0.23	+0.21		+0.11	+1.21
DINOv2/Aircraft	+0.65	+2.53	+1.88		+2.59	+2.19
ResNet-18/Aircraft	-0.01	+0.97	+0.98		+1.10	+2.05
mean	+0.22	+1.24	+1.02		+1.27	+1.95

Table 15. Δ vs. the Stage 1 probe (pp). "Flow contributes" is joint minus classifier-only.

Continuing to train the classifier alone moves it $+0.22$ pp, which independently confirms that Stage 1 left the probe at its ceiling rather than leaving that assumed. Everything the joint runs gain is therefore attributable to the flow. That the classifier's freedom goes unused can be seen three ways. Its learning rate spans $100\times$ ($10^{-5} \rightarrow 10^{-3}$) and every result lands between $+1.20$ and $+1.24$ pp. The resulting weight drift spans $50\times$ ($0.004 \rightarrow 0.214$) with no corresponding change in accuracy. And `delayed=50`, where the classifier barely moves at all (drift 0.004), scores identically to letting it move freely from epoch 0. Most telling of all, the classifier drifts more when trained alone (0.104) than jointly with the flow (0.039); once the flow is doing the work, there is nothing left for the classifier to do.

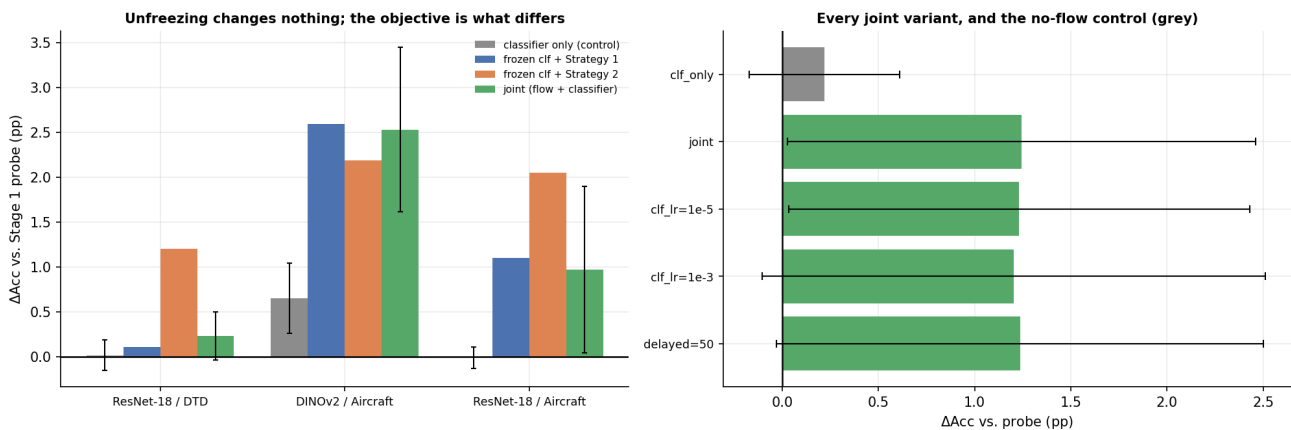


Figure 7. All settings against the probe (left), and every joint configuration alongside the classifier-only control in grey (right).

10 What Stage 3 answers

Does a flow in front of a frozen, trained classifier help?

Yes, but modestly: +0.1 to +2.6 pp. For contrast, Stage 2 found up to +22.3 pp applying the same idea in front of an untrained prototype classifier, and that contrast is the point, because Stage 2 predicted it. Stage 2's headroom result, that FM gain tracks trained probe minus untrained prototype with $\rho = 1.000$, says the flow recovers roughly what a trained classifier would have gained. In Stage 3 the classifier is trained, so that headroom is already spent. What remains is whatever a nonlinear transform can add on top of an already-fitted linear boundary, which turns out to be about a point.

Does classifier-guided training beat end-to-end rolled-out training?

Not on accuracy: +0.13 pp at $p = 0.80$ at matched learning rate. It is substantially more robust, significantly so ($p = 0.048$), and it improves in a higher fraction of runs, 91/99 against 67/81.

What limits both?

The frozen probe already classifies its own k-shot training set at 98.44–100.00%, with cross-entropy as low as 0.005. The signal both strategies chase is nearly exhausted before training starts, and following it further is memorisation. Both depend entirely on validation-accuracy checkpointing, Stage 1's own rule, to stop in time.

Limitations, stated plainly

- Three seeds give a t -test very little power. The p -values here are reported for completeness rather than leaned on; the more informative statement is how consistently the sign holds across the full sweeps (83% and 92% of runs).
- Three combinations cannot support a rule about which combinations benefit. DTD differs from the two Aircraft combinations in dataset, class count and training-set size simultaneously, so the ordering in step 02 is reported without an explanation attached.
- One K and one T , fixed for this stage; the K - and T -dependence measured in Stages 1 and 2 is not re-derived here.
- The weakest cell is Strategy 1 on DTD (+0.11 pp, one seed negative), and the step 05 margin diagnostic independently flags the same cell at 0.997×.

Reproducibility

52 tests pass across Stages 2 and 3. `test_stage3_formulas.py` re-derives each formula from its stated form and asserts that the implementation agrees; `test_probe_equivalence.py` asserts that Stage 3's probe loop matches Stage 1's on accuracy, predictions, selected epoch and every training curve. We validated the suite itself by breaking the implementation eight ways on purpose. The first run caught only five; the three misses were tests that re-implemented the formula inside the test file and therefore could not fail. After extracting the formulas into the public API, all eight are caught. Every figure and table here is produced by a tracked notebook, stored with its outputs and backed by a tracked CSV, and each step folder carries an `_artifact_manifest.csv` listing exactly what it wrote. Stages 1 and 2 are not modified: Stage 3 needed the probe's trained weights and a zero-initialised readout, both of which were added inside `cvlab3` rather than by editing published code, and git confirms that no Stage 1 or Stage 2 source file differs.